

CHAP 9: (1) 반복문 for

for Statement (for loop)

for문과 리스트, for문과 문자열, for문과 사전

for문과 range() 함수

중첩 for 루프 (Nested for loop)

for Statement (for loop)

- 반복이란?
 - 반복(iteration)은
동일한 문장을 여러 번 반복시키는 구조
- for반복문을 사용하여
 - 특정 작업을 일정 횟수 만큼 반복할 수 있다.
 - iterable자료형 객체(문자열,리스트,튜플,집합,사전,...)에서
원소들을 차례로 꺼내서 반복 작업을 한다.
 - 원소의 개수만큼 반복 실행

반복가능객체

{ 문자열
리스트
튜플, 집합, 사전
range(), keys(), values(), items() 등의 함수 값

for Statement (for loop)

- 만약 1000번 반복해야 한다면?

```
print("방문을 환영합니다!")  
print("방문을 환영합니다!")  
print("방문을 환영합니다!")  
print("방문을 환영합니다!")  
print("방문을 환영합니다!")  
...  
...  
print("방문을 환영합니다!")
```

1000번 복사해서 붙여넣기 ?



(출처: 두근두근 파이썬)

- 반복 구조를 사용한다.

```
for i in range(1000):  
    print("방문을 환영합니다!")
```

1000번 반복시키는 구조

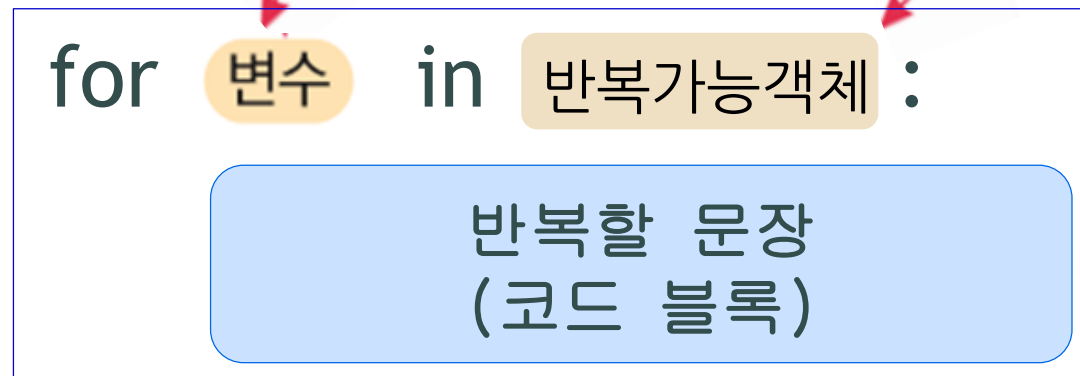
for Statement (for loop)

- for문은 일정한 횟수만큼 코드 블록을 반복 수행하는 구문
 - 반복 실행 제어:
반복가능객체(iterable객체)의 원소를 차례로 꺼내 **변수**에 할당하고 코드블록을 반복 실행. 더 이상 원소가 없으면 반복을 종료.
 - 원소의 개수만큼 반복 실행한다.
 - 반복가능객체**: 원소를 갖는 객체. 문자열, 리스트, 튜플, 집합, 사전, range(), keys(), values(), items() 함수의 값

- Syntax

반복될 때마다 차례로
원소를 하나 가져와서
변수에 할당.

원소를 갖는
자료형 객체



들여쓰기 해야 함

for Statement (for loop)

- Syntax

statementA

```
for 변수 in 반복가능객체 :
```

```
    statements  
    (code block)
```

statementB

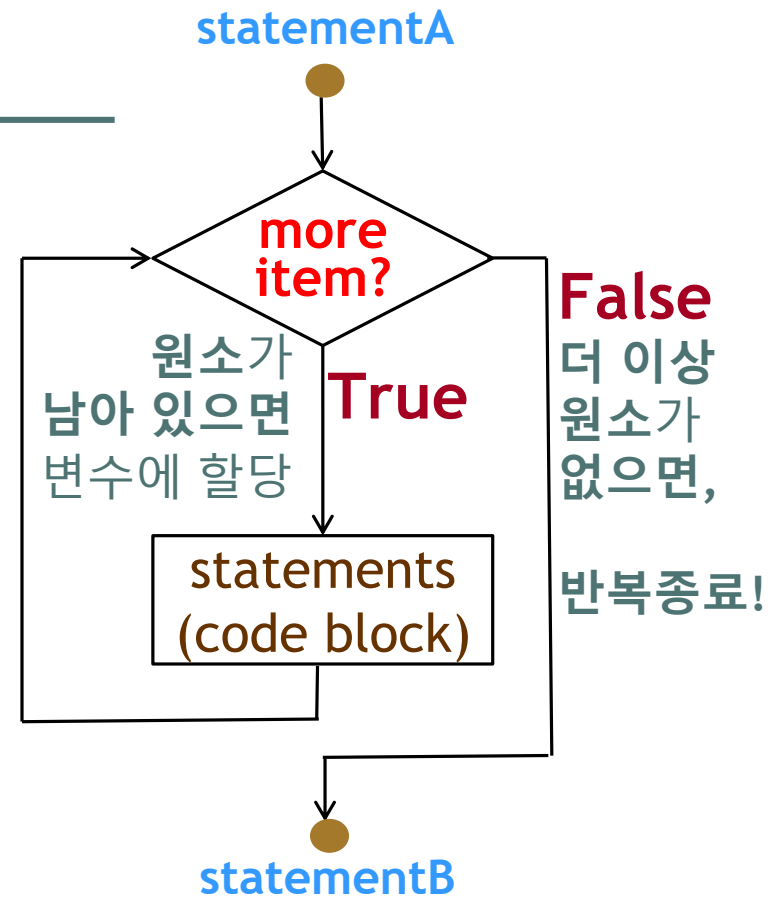
- 구성 요소 및 동작

- 반복가능객체(iterable object)

- 문자열, 리스트, 튜플, 집합, 사전 자료형 객체
- 또는 iterable 객체를 반환하는 함수.
range(), keys(), values(), items() ... 등등

- 변수 : 꺼내 온 원소가 할당되는 변수.

- for 문은 반복가능객체에서 각 원소를 차례로 꺼내 변수에 할당하고, 코드블록을 반복 실행. 더 이상 원소가 없으면 반복을 종료.



for문과 리스트

변수 **name**에
첫 번째 원소부터 차례대로 할당

- 리스트에 대한 반복

‘길동’

‘철수’

‘영희’

‘유신’

4회 반복

```
for name in ["길동", "철수", "영희", "유신"] :  
    print("안녕!")  
    print(name)  
  
print('♡♡')
```

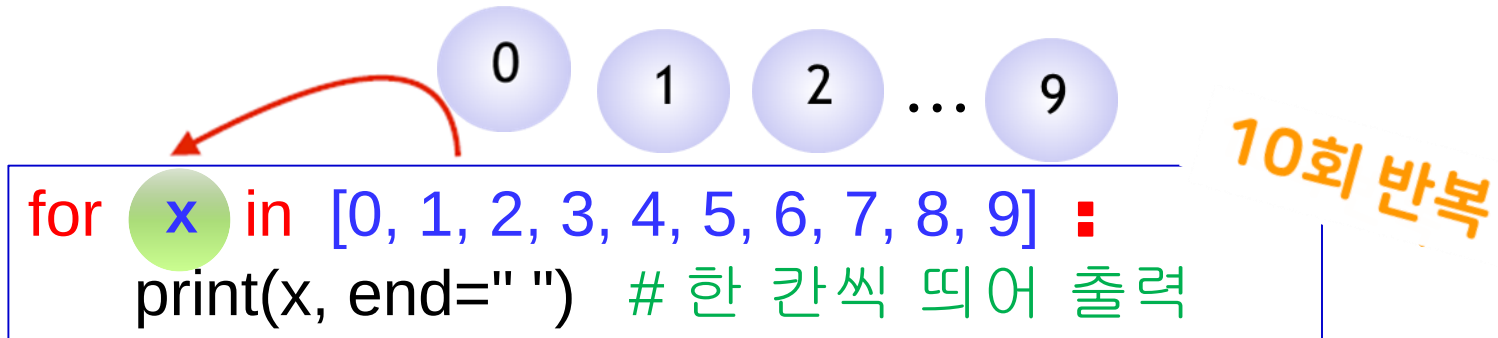
안녕!
길동
안녕!
철수
안녕!
영희
안녕!
유신
♡♡

😊 출력 화면

- ① 변수 **name**에 첫 번째 값 “길동” 할당
- ② 반복문장(코드블록) 실행 - 출력문 실행
- ③ 리스트에 있는 원소 값이 차례로 변수 **name**에 할당되고, 반복문장 실행을 반복한다.
- ④ “유신”까지 할당하여 반복코드를 수행 후 **for**문을 종료.
더 이상 가져오지 않은 원소가 없으므로 종료 함.

for문과 리스트

- (예) 리스트의 원소들을 한 줄에 출력하기



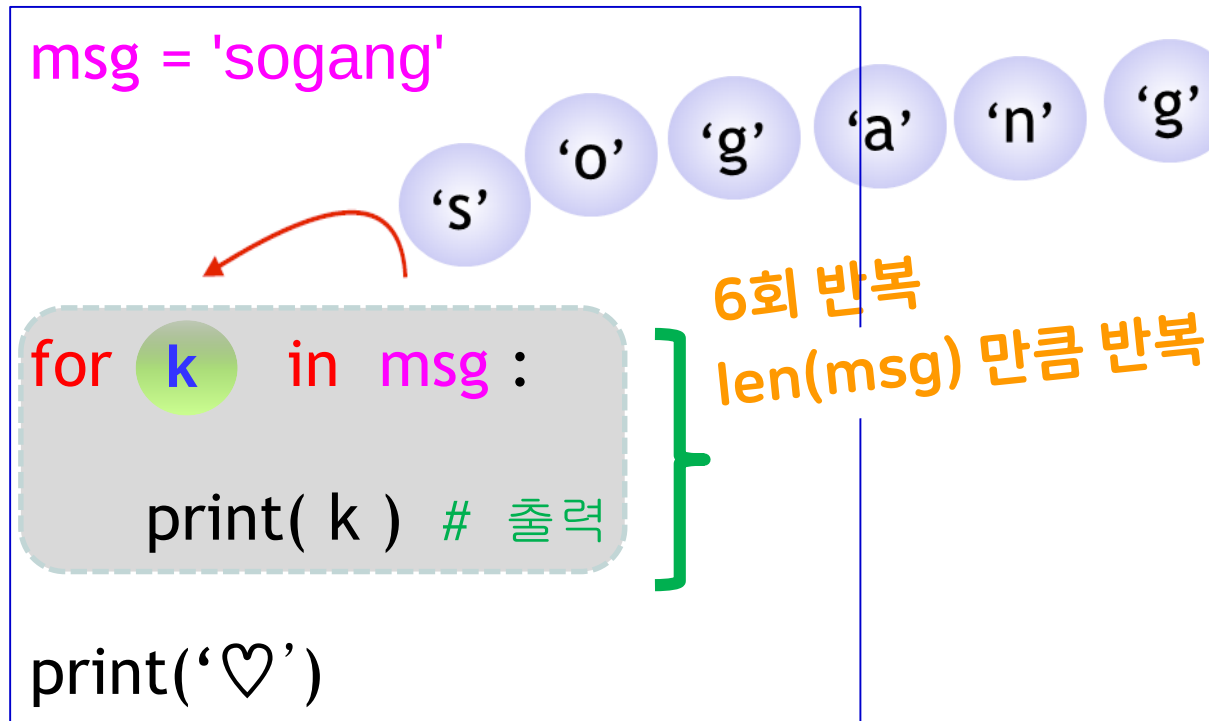
출력 화면

```
0 1 2 3 4 5 6 7 8 9
```

- ① 변수 x에 0 할당
- ② 출력
- ③ 리스트(대괄호 속 숫자들)에 있는 값이 차례로 변수 x에 할당되어 반복 코드블록을 실행.
- ④ 9까지 할당하여 반복 코드block 수행 후 for문 실행을 종료.

for문과 문자열

- 문자열 반복
 - 문자열은 원소가 있는 자료형이다.
따라서 문자열을 대상으로 반복문을 만들 수 있다.
- (예) 문자열의 문자(원소)를 각각 출력하기.
 - 문자열에 있는 문자를 하나씩 차례로 가져와 출력을 실행



😊실행 결과

s
o
g
a
n
g
♡

for문과 문자열

- 예) 1부터 10까지 카운트. 변수를 1씩 증가시키며 출력하기.
 - 카운트 변수를 0으로 초기화. 1씩 증가시키며 출력

cnt = 0 #카운트 변수

```
cnt = cnt + 1 #1  
print(cnt)
```

```
cnt = cnt + 1 #2  
print(cnt)
```

```
cnt = cnt + 1 #3  
print(cnt)
```

⋮

```
cnt = cnt + 1 #10  
print(cnt)
```

```
print('종료합니다')
```

#반복문 사용

```
cnt = 0 # 카운트 변수 초기화
```

```
for k in range(10):
```

```
    cnt = cnt + 1  
    print(cnt)
```

```
print('종료합니다.')
```

동일 작업을
10번 반복

😊 출력 화면

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
종료합니다
```

for문과 문자열

- 예) 문자열의 모음의 개수 집계하기. (모든 문자를 차례로 출력)
 - 특정 조건(모음일 때)에서만 카운트 변수 +1 증가

```
s = input('?? ')
cnt = 0

for k in s:          # 반복코드
    if k in 'aeiouAEIOU':
        cnt = cnt + 1
    print(k)

print("모음의 개수", cnt)
```

#for문장 다음 명령어

😊 출력 화면

```
?? ObeyToTheTruth Enter
O
b
e
y
T
o
T
h
e
T
r
u
t
h
모음의 개수 5
```

for문과 문자열

- 예) 문자열에서 모음의 개수를 집계하기. (모음 문자만 출력)

```
s = input('?? ')
cnt = 0

for k in s:
    if k in 'aeiouAEIOU':
        cnt += 1
        print(k)

print("모음의 개수", cnt)
```

반복코드

출력 화면

```
?? ObeyToTheTruth Enter
O
e
o
e
u
모음의 개수 5
```

for문과 사전

- 사전에 대한 반복
 - ‘키’ 값 들만 하나씩 가져와서 반복 처리를 한다

```
contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568',  
            'Lee': '010-123-4569' }
```

```
for name in contacts:  
    print(name)
```

“Kim”

“Park”

“Lee”

3회 반복

Kim
Park
Lee

출력 화면

- ① 사전객체에서 키를 하나 가져와 변수 **name**에 할당.
- ② 반복문장(여기서는 **print문**)을 실행
- ③ 다시 **for문** 처음으로 올라가 사전 객체에 있는 다른 키 값을 변수 **name**에 할당하고 반복문장(**print문**) 실행을 반복한다.
- ④ 더 이상 남아있는 키 항목이 없으면 **for문**을 종료.

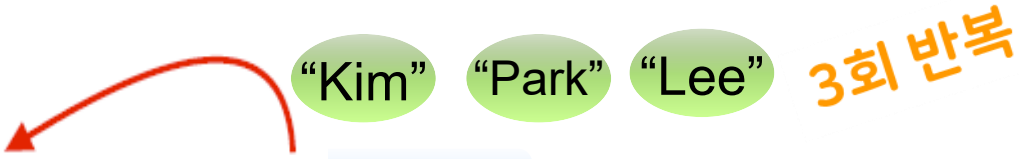
for문과 사전

- 사전에 대한 반복 - **Key값 순회하기**
 - 사전.keys()는 딕셔너리의 모든 Key를 모아서 리턴.
 - 사전.keys()의 반환 값을 가지고 반복문을 실행할 수 있다.

```
dict_keys(['Kim', 'Park', 'Lee'])
```

- 예) 모든 키 값 출력하기

```
contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568',  
            'Lee': '010-123-4569' }
```



```
for k in contacts.keys() : # Key값 순회  
    print(k)
```

Kim
Park
Lee

출력 화면

for문과 사전

- 사전에 대한 반복 - **Key값 순회하기**
 - (예) 키 값으로 대응되는 **value** 가져오기

```
contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568',  
            'Lee': '010-123-4569' }
```



```
for k in contacts.keys() :    # Key값 순회  
    print( contacts[k] )    # Key값으로 value 출력
```

010-123-4567

010-123-4568

010-123-4569

출력 화면

for문과 사전

- 사전에 대한 반복 - **Value값 순회하기**
 - **dict.values()**는 딕셔너리의 모든 Value를 모아서 리턴.
 - **dict.values()**의 반환 값을 가지고 반복문을 실행

```
dict_values(['010-123-4567','010-123-4568', '010-123-4569'])
```

```
contacts = {'Kim': '010-123-4567', 'Park': '010-123-4568',  
            'Lee': '010-123-4569' }
```

```
for k in contacts.values() : # values값 순회 3회 반복  
    print(k)
```

```
010-123-4567  
010-123-4568  
010-123-4569
```

출력 화면

for문과 사전

- 사전에 대한 반복 - **Key와 Value 쌍을 순회하기**
 - **dict.items()**는 딕셔너리의 원소(**key와 value 쌍**)를 튜플로 묶고, 이들을 모아서 반환.
 - 예) `dict_items([('Korean', 80), ('Math', 90), ('English', 80)])`
 - `dict.items()`의 반환 값으로 반복문을 실행할 수 있다.
 - 예1) 튜플로 묶여진 값을 변수 하나로 받음. 즉 튜플로 받는 경우

```
scores= {'Korean': 80, 'Math': 90, 'English': 80}
```

#원소 순회

```
for pair in scores.items():  
    print(pair)
```

('Korean', 80)

```
dict_items([('Korean', 80),  
            ('Math', 90), ('English', 80)])
```

#pair은 튜플을 할당받음
#pair = (key, value)

```
('Korean', 80)  
('Math', 90)  
('English', 80)
```

실행 화면

for문과 사전

- 사전에 대한 반복 - **Key와 Value 쌍을 순회하기**
 - dict.items()의 반환 값으로 반복문을 실행.
 - dict_items([('Korean', 80), ('Math', 90), ('English', 80)])
 - 예2) key와 vlaue 쌍의 **튜플**을 각각 다른 변수로 받는 경우

```
scores= {'Korean': 80, 'Math': 90, 'English': 80}
```

#원소 순회

```
for c,mark in scores.items():  
    print(c,mark)
```

('Korean', 80)

```
dict_items([('Korean',80),  
            ('Math',90),('English',80)])
```

#튜플 unpacking

key와 value를 각각 따로 받음

#c,mark = (key, value)

Korean 80
Math 90
English 80

실행 화면

for문과 사전

- 반복문을 사용한 사전 이용하기.
 - value가 리스트 객체인 경우, 참조를 복사.
동일한 객체를 다른 이름으로 참조하게 된다.

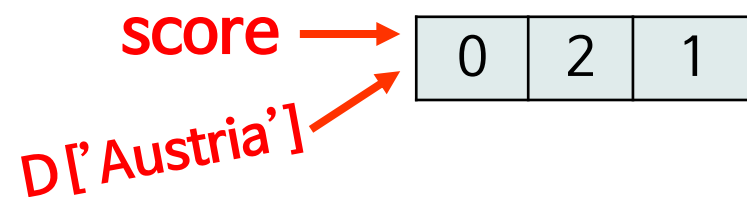
첫번째 반복에서, 변수score가 참조.

```
D = { 'Austria': [0,2,1], 'Canada':[4,8,5], 'China':[5,0,1] }  
  
for c, score in D.items(): # key와 value를 각각 전달 3회 반복  
    print('{} 금{} 은{} 동{}'.format(c, score[0],score[1],score[2]) )  
  
print()
```

☺ 출력 화면

Austria 금0 은2 동1
Canada 금4 은8 동5
China 금5 은0 동1

1번째 반복 때에



for문과 사전

- 반복문을 사용한 사전 이용하기.
 - value가 리스트 객체인 경우, 참조를 복사.
동일한 객체를 다른 이름으로 참조하게 된다.

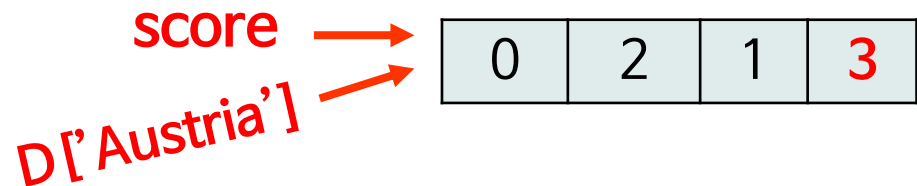
D['Austria'], score가 참조.

```
D = {'Austria': [0,2,1], 'Canada':[4,8,5], 'China':[5,0,1]}
```

```
for score in D.values(): # value를 차례로 전달
    hap= score[0]+score[1]+score[2] # hap=sum(score)
    score.append(hap)
print(D)
```

3회 반복

1번째 반복 때에



실행 화면

```
{ 'Austria':[0,2,1,3], 'Canada':[4,8,5,17], 'China':[5,0,1,6]}
```

for문과 range() 함수

- range() 함수

`range( end )` #start, step 생략. 디폴트 값 사용

`range( start , end , step )`

- start부터 end-1까지 step의 간격으로 정수들을 생성한다.
 - start는 생략되면 0. step은 생략되면 1.
 - end는 생략될 수 없다!!!
- range() 함수는 특정 구간의 정수들을 생성.
 - 일련의 정수 sequence를 생성. (list, tuple, set 등으로 변환 가능)
 - 주로 for 반복문과 함께 사용된다
 - for문의 단순 반복 회수 제어에 사용
 - 리스트의 인덱스 번호로 사용할 수 있다.

for문과 range() 함수

- range(**end**)
 - 0부터 **end-1**까지 **1씩 증가**하는 정수 **sequence** 객체 생성.
- range(start, end)
 - start부터 **end-1**까지 **1씩 증가**하는 정수 **sequence** 객체 생성.

● (예)

range(5)



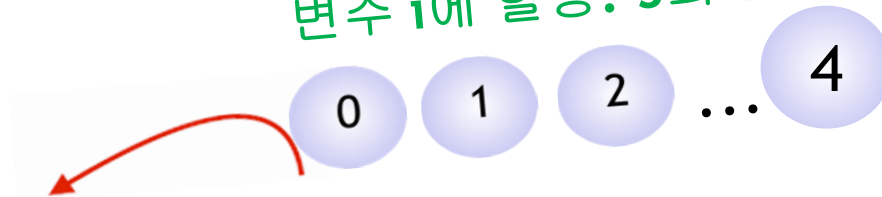
- end만 지정. 0,1,2,3,4 정수 **sequence** 생성
- 5회 반복에 사용할 수 있다
- 크기가 5인 리스트의 인덱스로 사용

```
>>> L = list( range(5) )   # list()를 통해 리스트 객체로 변환
>>> print(L)
[0, 1, 2, 3, 4]
```

for문과 range() 함수

- range()가 생성한 정수 sequence로 **반복 회수를 제어**
 - 생성된 sequence의 값을 차례로 변수에 할당하여 코드 block을 반복 실행한다.
- 예) **5회 반복**
 - range(5) 0~4까지 정수 sequence를 생성

변수 i에 할당. 5회 반복



```
for i in range(5):  
    print("방문을 환영합니다!")
```

😊 출력 화면

방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!

for문과 range()함수

- range()에서 생성한 정수 sequence를 index로 사용
- 예) range(len(L))으로 인덱스 생성

```
>>> L = [ 10, 20, 30, 40, 50 ]
```

```
>>> len(L)
```

5

```
>>> range( len(L) ) # 0~4까지 정수 sequence 생성.  
range(0,5).          인덱스로 사용
```

```
>>>
```

생성된 정수들을
index로 사용



for문과 range() 함수

- 예) 리스트의 원소 가져오기
 - range()에서 생성한 정수 sequence를 index로 사용

```
L = [ 10, 20, 30, 40, 50 ]
```

변수 i에 할당. 인덱스로 사용

```
for i in range( len(L) ) : # i에 차례대로 0,1,2,3,4 할당  
    print( L[i] )
```

```
print(L)
```

😊 출력 화면

```
10  
20  
30  
40  
50  
[10,20,30,40,50]
```


for문과 range() 함수

- 예) 리스트의 원소 값 수정하기
 - range()에서 생성한 정수 sequence를 index로 사용

```
L = [ 10, 20, 30, 40, 50 ]
```

변수 i에 할당. 인덱스로 사용

```
for i in range( len(L) ) : # i에 차례대로 0,1,2,3,4가 할당.  
    L[i] = L[i] * 10      # L[i] 값을 수정. 원소 수정.
```

```
print(L)
```

😊 출력 화면

```
[ 100, 200, 300, 400, 500 ]
```

for문과 range() 함수

- 예) 1부터 10까지의 합 구하기. $a = 1+2+3+4+5+....+9+10$

```
a = 0 # 초기화 필요
for i in range( 1, 11 ) :
    a = a + i
print(a) # 55
```

1~10까지
정수sequence

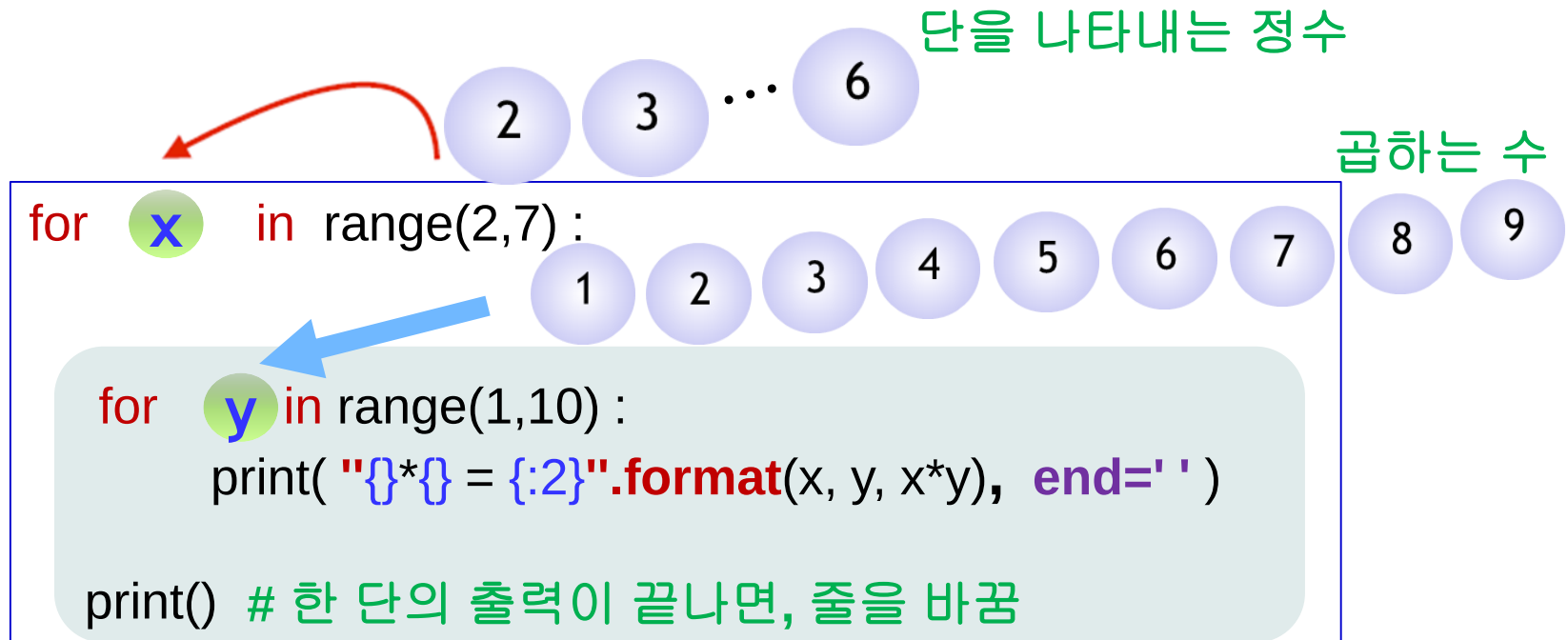
	i의 값	a의 값
1번째 반복	1	0+1
2번째 반복	2	0+1+2
3번째 반복	3	0+1+2+3
...	...	...
10번째 반복	10	0+1+2+3+ +9+10

```
a=0
a=a+1
a=a+2
a=a+3
a=a+4
a=a+5
a=a+6
a=a+7
a=a+8
a=a+9
a=a+10
```

$a = a + i$

Nested for loop

- 예) 구구단 출력하기. 2단부터 6단까지 출력.
 - 각 단마다 1~9까지 정수로 곱함

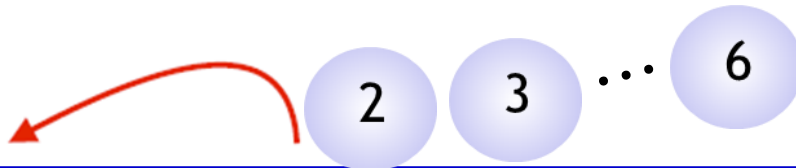


☺ 출력 화면

```
2*1= 2 2*2= 4 2*3= 6 2*4= 8 2*5=10 2*6=12 2*7=14 2*8=16 2*9=18  
3*1= 3 3*2= 6 3*3= 9 3*4=12 3*5=15 3*6=18 3*7=21 3*8=24 3*9=27  
4*1= 4 4*2= 8 4*3=12 4*4=16 4*5=20 4*6=24 4*7=28 4*8=32 4*9=36  
5*1= 5 5*2=10 5*3=15 5*4=20 5*5=25 5*6=30 5*7=35 5*8=40 5*9=45  
6*1= 6 6*2=12 6*3=18 6*4=24 6*5=30 6*6=36 6*7=42 6*8=48 6*9=54
```

Nested for loop

단을 나타내는 정수



```
for x in range(2,7):
```

곱하는 수

```
print( "{}*{} = {:2}".format(x, 1, x*1) , end=' ' )  
print( "{}*{} = {:2}".format(x, 2, x*2) , end=' ' )  
print( "{}*{} = {:2}".format(x, 3, x*3) , end=' ' )  
print( "{}*{} = {:2}".format(x, 4, x*4) , end=' ' )  
print( "{}*{} = {:2}".format(x, 5, x*5) , end=' ' )  
print( "{}*{} = {:2}".format(x, 6, x*6) , end=' ' )  
print( "{}*{} = {:2}".format(x, 7, x*7) , end=' ' )  
print( "{}*{} = {:2}".format(x, 8, x*8) , end=' ' )  
print( "{}*{} = {:2}".format(x, 9, x*9) , end=' ' )
```

```
print() # 한 단의 출력이 끝나면, 줄을 바꿈
```

비교

27쪽과 비교!!